Kuku Promo Finder: Find the Promos (Problem 1)

Author: Rakshit · **Repo:** `run.py` (engine), `app/` (Next.js viewer) · **Run:** `outputs/promos.meta.json`

1. Problem and scope

An editor watches every episode to pick 20-60s promos by hand, and new shows ship weekly. The tool reads timestamped screenplays and returns ranked promos: episode, every segment's start/end, hook, cliffhanger, one line on why it works. Assumptions: the screenplay is the only input (no video), so "visual" judgements are limited to what stage directions say; a promo is one continuous cut from one episode (cross-episode splicing exists but is experimental and off by default, because nothing in the code can tell whether two scenes cohere).

2. Approach: what is AI, what is code

Step	Owner	Why
Parse 50 episodes into 3,727 timed beats	Code (`src/parser.py`)	Timestamps must be exact
Find candidate windows, tag hook/cliffhanger, score 5 dimensions, say why	LLM (`gpt-6-luna`, one call per episode)	Needs dramatic judgement
Enforce 20-60s, hook beat starts within 3s, every cut on a real beat edge, hook/cliffhanger quotes exist verbatim, cliffhanger is the last 6s	Code (`src/validation.py`)	Models invent timestamps and quotes, so they are checked, not trusted. LLM cut points within 0.5s of a real beat edge are snapped; the rest are dropped (4 of 48 proposals were)
Adversarial audit of each candidate: the model reads the actual clip screenplay (camera beats included) plus the tag definitions and returns pass/fail with one of 7 flaw types	LLM	Five failure modes: resolution leak, context deficit, visual dependency, slow burn, false reading (+ wrong tag)
Rank: score = 0.30 hook + 0.25 cliffhanger + 0.20 injustice + 0.15 clarity + 0.10 payoff, times taxonomy weights, a 0.82x penalty per repeated hook type and 0.85x per repeated episode; audit-passing promos rank above failing ones	Code	Transparent and repeatable

The hook/cliffhanger taxonomy is config (`config/taxonomy.yaml`, 10 hooks, 6 cliffhangers, with `requires` / `do_not_tag_if`); a new show needs one profile file (`config/shows/<show>.yaml`) and `--show <show>`. Offline keyword mode (`--provider heuristic`) exists as a smoke test only. It is labelled as such in every output.

3. Result (raw outputs in `outputs/`)

Run `gpt-6-luna`, 50 episodes: 48 proposals, 44 valid, 42 after de-duplication, **23 passed the model audit (55%), 19 failed**. Top 15 are all audit-passing by construction. 13 episodes yielded no candidate (including 6 and 8, which are quiet scenes). Rank 1 is Ep 41, 34.4s: nobles sneer "Look at her." at the chained protagonist, then a second segment jumps to the cathedral breach and ends on armed figures in the smoke. Rank 2 is Ep 15, 32.0s: a thug shoves Juliana in the first second, a club is raised over her mother's plants, Alexander storms in and the cut lands as he faces the thugs. They rank first on the weighted score; no human editor has reviewed this run, so treat the order as the tool's opinion, not a verified one.

4. Where it picks badly (real runs)

- **The offline audit was worthless.** The earlier committed output (keyword engine) passed 87/87 of its own audit because its fail thresholds were unreachable from its constant scores. The same 15 promos run through the LLM auditor: **2 pass, 13 fail.** The keyword audit is now labelled "rule lint, not a quality judgement".
- **The model audit disagrees with the extractor often:** of 19 failures, 13 are `wrong_tag` (e.g. Ep 5: "Choose your next words carefully" was tagged as a withheld threat but is delivered in full), 4 `context_deficit`, 2 `resolution_leak`.
- **Taxonomy fit:** 10 of the top 15 carry a model-invented `new:` hook type (e.g. `new:poisoning_reveal`), i.e. the 10 hooks do not cover this show's best moments; they get a neutral 0.90 weight.
- **Self-grading:** extractor and auditor are the same model in this run. `--audit-model` allows an independent judge; I did not run one.
- **Unmeasured:** there is no hand-picked ground truth, so recall and precision are unknown. `eval/recall.py` computes them once `eval/ground_truth.json` is committed.
- Earlier failures, now fixed: cliffhanger quotes that appeared 20s before the cut ended; truncated quotes; `--- SCRIPT END ---` text inside a quote; silent fallback to keyword output when an API call failed.

5. Human in the loop

The editor reviews `outputs/PROMOS.md` (or the Next.js viewer): every promo shows the audit verdict, flaw type and risks, and failing candidates sit below all passing ones. There are no approve/trim controls; the editor trims in their own tool using the exact timestamps.

6. Cost at volume (measured, 50 episodes, one show)

92 calls (50 extraction + 42 audit), **250.7s wall** with 8 parallel workers, **546,585 tokens** (376,060 in, 170,525 out of which 138,056 reasoning). At `gpt-6-luna` $0.10 / $0.50 per M tokens (rates from third-party listings, not OpenAI's page) that is about **$0.12**; the same tokens at Gemini 3.8 Flash's introductory $0.75 / $3.75 (from search results of Google's pricing page,

unverified) would be about **$0.92**. Re-runs with unchanged prompts cost nothing (disk cache), and failed calls are retried with backoff. Weekly new shows therefore cost on the order of $1 each, and a human spends their time on review, not discovery.

7. Tools used

Claude (via Claude Code) wrote and reviewed most of the code, tests and this document; GPT `gpt-6-luna` is the runtime model. See `AGENTS.md` for logged AI mistakes with artifacts.